

Brother Money - Protection Engine Specification (v2)

Purpose

The Protection Engine determines how much of the user's money should already be protected today.

Its purpose is not to create budgets.

Its purpose is to answer:

How much money can I safely spend right now without harming future obligations?

The engine runs whenever:

- The app opens
- Cash is updated
- A transaction is logged
- A bill is modified
- A savings goal is modified

Core Philosophy

Brother Money is a financial pacing system.

The engine does not ask:

How much money should eventually be saved?

Instead it asks:

Based on where we are in the cycle, how much money should already be protected by now?

This creates recommendations that adapt naturally over time.

Engine Inputs

Current Cash

Source:

Latest Cash Snapshot

Example:

Current Cash

\$500

Current Cash is always considered the source of truth.

If transaction history disagrees with current cash:

Current Cash wins.

Protection Layers

Money is protected in the following order:

1. Bills
2. Savings
3. Emergency Buffer

Remaining money becomes Safe Spend.

Bill Funding Model

Problem

Traditional budgeting apps reserve the entire bill amount immediately.

Example:

Rent

\$300

Due in 30 days

Protected:

\$300

This often makes users feel artificially poor.

Funding Window

Every bill contains:

```
type Bill = {  
  id: string;  
  
  name: string;  
  
  amount: number;  
  
  lastPaidDate: string;  
  
  nextDueDate: string;  
  
  active: boolean;  
};
```

Example:

Rent

Amount:
\$300

Last Paid:
June 1

Next Due:
July 1

Funding Window:

June 1 → July 1

Required Protection

The engine calculates:

Cycle Progress

=

Days Elapsed

/

Total Cycle Days

Then:

Required Protection

=

Bill Amount

×

Cycle Progress

Example

Rent:

Amount

\$300

Cycle:

June 1 → July 1

Today:

June 16

Progress:

15 / 30

=

50%

Required Protection:

$300 \times 50\%$

=

\$150

Meaning:

By today, \$150 should already be protected.

Bill Status

Every bill receives a status.

Ahead

Required Protection:

\$150

Available Protection:

\$200

Result:

Ahead

On Track

Required Protection:

\$150

Available Protection:

\$150

Result:

On Track

Behind

Required Protection:

\$150

Available Protection:

\$90

Result:

Behind \$60

Missed Due Dates

If:

Today > Next Due Date

Then:

Required Protection

=

Full Bill Amount

The bill becomes immediately overdue.

Bill Payment

When a bill is paid:

User can:

Log Expense

or

Mark Paid

The engine updates:

Last Paid Date

=

Today

and

Next Due Date

=

Next Billing Cycle

The funding cycle restarts.

Savings Funding Model

Savings follow the same logic as bills.

Example Goal

Save

\$80

Per Month

Funding Window:

June 1 → July 1

Today:

June 16

Progress:

50%

Required Protection:

$80 \times 50\%$

=

\$40

Meaning:

The user should already have protected \$40 toward savings.

Savings Status

Savings can also be:

- Ahead
- On Track
- Behind

Using the same rules as bill protection.

Emergency Buffer

Emergency Buffers are different.

They are protected immediately.

Example:

Emergency Buffer

\$100

Protected:

\$100

Always.

No accrual.

No funding cycle.

Total Protection Calculation

Example:

Current Cash:

\$500

Required Bill Protection:

\$150

Required Savings Protection:

\$40

Emergency Buffer:

\$100

Total Protected:

\$290

Safe Spend Formula

Safe Spend

=

Current Cash

-

Required Bill Protection

-

Required Savings Protection

-

Emergency Buffer

Example:

500

-

150

-

40

-

100

=

210

Result:

Safe Spend

\$210

Negative Safe Spend

Example:

Current Cash

\$200

Total Protected:

\$250

Result:

Safe Spend

-\$50

User Message:

You're currently \$50 behind your protected commitments.

No judgement.

Just reality.

Purchase Approval Engine

User enters:

Shoes

\$40

Decision Rule:

Purchase Cost

$\leq$

Safe Spend

Result:

Safe To Buy

Otherwise:

Not Yet

Purchase Feedback

Example:

Safe Spend:

\$80

Purchase:

\$40

Result:

Safe To Buy

After purchase:

\$40 remains available.

Example:

Safe Spend:

\$15

Purchase:

\$40

Result:

Not Yet

Need \$25 more before this purchase becomes safe.

Confidence Engine

Purpose:

Measure confidence in the guidance.

Not confidence in the calculations.

Confidence in the underlying data.

Cash Freshness

Updated Today:

100%

Updated 3 Days Ago:

90%

Updated 7 Days Ago:

75%

Updated 14 Days Ago:

50%

Logging Consistency

Recent transaction activity increases confidence.

Long periods without activity reduce confidence.

Confidence Output

Example:

Confidence

92%

Message:

Guidance is based on recent cash updates.

Example:

Confidence

55%

Message:

Update your current cash for more accurate guidance.

Brother Advice Rules

No AI required.

Simple deterministic rules.

Low Safe Spend

Safe Spend < 10

Message:

Bills are approaching. Spend carefully.

Ahead On Savings

Savings Status = Ahead

Message:

You're ahead of your savings target.

Large Cushion

Safe Spend > 25% of Current Cash

Message:

You're in a comfortable position right now.

Behind Commitments

Safe Spend < 0

Message:

Focus on rebuilding protected money first.

Final Engine Output

The Protection Engine produces:

Current Cash

Protected Bills

Protected Savings

Emergency Buffer

Total Protected

Safe Spend

Confidence

Brother Advice

Every recommendation inside Brother Money originates from these values.

The engine exists to answer one question:

How much money is truly available today?